



VIETTEL AI RACE

LLM Inference Optimization Challenge — Vòng 1: Tổng hợp thông tin, kết quả phân tích trace và chiến lược đạt Top 1

Cập nhật: 05/07/2026 — còn khoảng 25
ngày đến hạn chót vòng 1 (30/07/2026)

Phần 1. Toàn bộ thông tin từ Ban tổ chức

1.1. Bối cảnh và nhiệm vụ

Cuộc thi **LLM Inference Optimization**

Challenge mô phỏng bài toán thực tế của các đội ngũ hạ tầng AI doanh nghiệp: phục vụ mô hình ngôn ngữ lớn với thông lượng cao, độ trễ thấp, độ chính xác ổn định trên tài nguyên GPU hữu hạn. Ở vòng 1 (sơ loại, diễn ra từ 02/07/2026 đến 30/07/2026), thí sinh phải triển khai và tối ưu một LLM inference server cho mô hình

Qwen/Qwen3.5-2B, xử lý một file trace gồm **120 request** mô phỏng traffic production. Toàn bộ quá trình chấm được thực hiện tự động: thí sinh serve endpoint, hệ thống của BTC bắn benchmark trực tiếp vào endpoint đó. Tất cả các đội chạy cùng một file trace với cùng arrival timestamp, nên sân chơi hoàn toàn đồng nhất — thứ hạng chỉ phụ thuộc vào chất lượng serving stack.

1.2. Hạ tầng và mô hình

Môi trường chấm là **1 instance MIG H200 với 18GB VRAM, 3 core CPU và 8GB RAM**, chạy Ubuntu 22.04 LTS và CUDA 12.x. Đây là ràng buộc rất chặt: chỉ một lát cắt nhỏ của H200, CPU và RAM đều hạn hẹp. Trọng số model tải từ HuggingFace Hub theo hash cố định do BTC công bố.

Về mô hình, đề bài mô tả Qwen3.5-2B là "Dense Transformer, BF16 gốc, Apache 2.0". Tuy nhiên qua tra cứu thực tế, đây là **kiến trúc lai (hybrid)**: các layer Gated DeltaNet (linear attention, state kích thước cố định) xen kẽ với full attention theo pattern lặp, context gốc lên tới 262.144 token. Chữ "Dense" trong đề nhiều khả năng chỉ có nghĩa "không phải MoE". Đặc điểm này ảnh hưởng trực tiếp đến chiến lược: KV cache thật sự tăng theo độ dài chuỗi chỉ nằm ở một phần nhỏ số layer, nên footprint bộ nhớ theo context nhỏ hơn nhiều so với Transformer thuần — nhưng đồng thời cũng đặt dấu hỏi về mức độ hỗ trợ prefix caching của các engine đối với kiến trúc lai này (cần kiểm chứng thực nghiệm, xem Phần 3).

1.3. Cơ chế tính điểm

Điểm cuối cùng là **phép nhân** giữa hiệu năng và chất lượng:

$$\text{Score} = 100 \times \text{ERS} \times f(\Delta)$$

ERS (Effective Request Score) là trung bình cộng điểm của toàn bộ 120 request. Mỗi request nhận 0 điểm nếu lỗi, timeout hoặc trả về 0 token; nếu xử lý thành công thì điểm là $w \cdot s_{\text{ttft}} + (1-w) \cdot s_{\text{tpot}}$, trong đó mỗi thành phần được nội suy liên tục giữa ngưỡng lý tưởng (Floor) và ngưỡng loại bỏ (Ceiling), rồi lũy thừa γ :

$$s = [\text{clamp}((C - x)/(C - F), 0, 1)]^\gamma$$

Riêng vòng 1, thành phần TPOT dùng **TPOT trung bình** của toàn bộ output trong request. Tham số cụ thể vòng 1:

Tham số	Giá trị	Nhận xét
F_ttft / C_ttft	100 ms / 1500 ms	Cửa sổ rộng (15×)
F_tpot / C_tpot	20 ms / 45 ms	Cửa sổ rất hẹp (2,25×) — ràng buộc gắt
γ	2	Điểm dốc mạnh: càng gần Floor, mỗi ms cải thiện càng có giá trị
w	0,5	TTFT và TPOT quan trọng ngang nhau

Accuracy Gate được chấm độc lập bằng 100 câu hỏi cố định trích từ GPQA Diamond. Baseline reference là model BF16 gốc đạt 0,4. Độ sụt giảm $\Delta = 0,4 - \text{accuracy}$ của đội. Hàm phạt $f(\Delta)$ bằng 1,0 nếu $\Delta \leq 0,10$ (tức accuracy còn $\geq 0,30$ thì hoàn toàn không bị phạt); giảm tuyến tính về 0 trong khoảng $0,10 < \Delta < 0,16$; và bằng 0 nếu $\Delta \geq 0,16$ (accuracy $\leq 0,24$, xấp xỉ mức đoán ngẫu nhiên của trắc nghiệm 4 lựa chọn). Vì là phép nhân, tụt accuracy quá ngưỡng đồng nghĩa mất trắng toàn bộ điểm bất kể hệ thống nhanh đến đâu, và ngược lại.

1.4. Quy trình nộp bài

Thí sinh đóng gói toàn bộ giải pháp thành Docker Image, đẩy lên Docker Hub ở chế độ public, rồi nộp file docker-compose.yml qua Portal của BTC. Hệ thống tự động pull image, dựng container trên instance MIG, chạy healthcheck rồi benchmark; kết quả và log trả về trong khoảng 15 phút, leaderboard cập nhật tự động. BTC cung cấp baseline là image vllm/vllm-openai:v0.22.1 kèm compose mẫu (đáng chú ý: max-model-len=262144, gpu-memory-utilization=0.95, enable-prefix-caching đã bật sẵn).

1.5. Luật chống gian lận

Nghiêm cấm: hardcode đáp án của probe subset; pre-compute response cho các request nằm trong trace; gọi network external từ inference server khi serving (không call API ngoài, không pull model động); chỉnh sửa tokenizer; thay đổi arrival timestamp hoặc cấu hình concurrency; dùng account phụ để leak hidden trace. Vi phạm dẫn đến void submission hoặc loại khỏi cuộc thi. Hệ quả thực dụng quan trọng nhất của các luật này đối với chiến lược: **trọng số phải được nướng sẵn trong image** (vì cấm pull runtime), và **không được "làm nóng" cache bằng chính nội dung trace trước giờ chấm** — hành vi đó rơi vào định nghĩa pre-compute.

Phần 2. Quá trình phân tích file trace-round1.jsonl

Quá trình phân tích được thực hiện qua ba chương trình xây dựng nối tiếp nhau, mỗi bước đào sâu một giả thuyết nảy sinh từ bước trước.

2.1. Bước 1 — Phân tích tổng quát (analyze_trace.py)

Chương trình đầu tiên quét toàn bộ 120 dòng và cho ra bức tranh nền. Timestamp tăng đơn điệu tuyệt đối. Khoảng cách giữa các request (inter-arrival time) có trung vị chỉ 25 ms nhưng giá trị lớn nhất lên tới 4.525 ms, hệ số biến thiên $CV = 4,22$ — dấu hiệu burst rõ rệt, hoàn toàn không phải phân phối Poisson tự nhiên. QPS trung bình toàn file là 4,71 nhưng đỉnh theo cửa sổ 1 giây đạt 20 req/s. Về độ dài, prefill trung bình xấp xỉ 18.900 token (đếm bằng tiktoken cl100k_base làm proxy), dao động từ 12.919 đến 24.936 token, trong khi max_tokens cố định 200 cho mọi request — tỷ lệ prefill:decode xấp xỉ 95:1, tức bài toán nghiêng hẳn về phía xử lý context dài. Toàn bộ request dùng chung một model, temperature=0 và seed cố định. Cấu trúc hội thoại chia đúng 6 mức số message (2, 4, 6, 8, 10, 12), mỗi mức đúng 20 request; cả 120 request dùng chung một system prompt duy nhất. Nội dung message là văn bản sinh tự động dạng "word-salad" với từ khóa lặp thuộc domain hệ thống phân tán/ML.

2.2. Bước 2 — Kiểm chứng cấu trúc burst (verify_burst_groups.py)

Giả thuyết đặt ra: các request đến theo từng cụm, mỗi cụm ứng với một mức độ dài hội thoại. Chương trình tự động phát hiện ranh giới nhóm dựa trên bước nhảy IAT (ngưỡng tự tính 336,3 ms, không hard-code) và xác nhận: đúng **6 nhóm, mỗi nhóm 20 request**, IAT bên trong nhóm chính xác tuyệt đối 25,00 ms (min = max = mean), khoảng nghỉ giữa các nhóm chính xác tuyệt đối 4.525 ms, và số message theo thứ tự nhóm là 2 → 4 → 6 → 8 → 10 → 12, đồng nhất hoàn toàn trong từng nhóm. Độ chính xác tuyệt đối đến từng ms khẳng định trace được sinh bằng script có kiểm soát, không phải capture từ traffic thật.

2.3. Bước 3 — Kiểm chứng tính liên tục prefix (check_prefix_continuity.py)

Câu hỏi then chốt: 6 nhóm là 6 bộ nội dung độc lập, hay là cùng những hội thoại được "cắt" ở các độ dài khác nhau? Kết quả kiểm chứng cho câu trả lời dứt khoát: khi ghép request thứ *i* của nhóm ngắn với request thứ *i* của nhóm dài kế tiếp, **tỷ lệ khớp là 100% ở mọi vị trí message, không một ngoại lệ nào trong cả 100 cặp so sánh**. Độ dài prefix chung tính bằng token tăng dần qua các cặp nhóm:

Cặp nhóm	Token prefix chung trung bình
2→4	12.932
4→6	15.808
6→8	18.683
8→10	21.557
10→12	24.432

Mỗi "vòng hội thoại" bổ sung khoảng 2.875 token mới (một message assistant có sẵn trong trace cộng một message user mới).

2.4. Bức tranh tổng hợp — bản chất thật của workload

Ghép ba bước lại, trace này không phải 120 request rời rạc mà là **20 phiên hội thoại (session) chạy song song, mỗi phiên 6 lượt (turn), các lượt đến đồng bộ theo từng round**: request ở vị trí thứ i trong mỗi nhóm chính là lượt tiếp theo của session thứ i . Mỗi round là một burst 20 request dồn trong khoảng 475 ms (tốc độ tức thời trong burst xấp xỉ 40 req/s), giữa các round có khoảng nghỉ 4,5 giây mô phỏng "think time" của người dùng. Prefix dùng chung trong cùng session tăng dần từ gần 13 nghìn đến hơn 24 nghìn token — gần như toàn bộ prefill của request. Đây là kịch bản kinh điển để đánh giá **session-aware prefix caching / KV-cache reuse**: hệ thống tốt phải nhận ra lượt sau chia sẻ phần đầu với lượt trước và chỉ tính phần mới, thay vì prefill lại từ đầu. Lưu ý kỹ thuật: các con số token ở trên dùng tokenizer cl100k_base làm proxy; trước khi tính ngân sách bộ nhớ chính thức cần đếm lại bằng tokenizer thật của Qwen3.5-2B.

Phần 3. Chiến lược đạt Top 1

3.1. Đọc lại luật chơi qua lăng kính trace: cuộc thi thực chất là cuộc đua gì?

Kết hợp công thức điểm với cấu trúc trace, có thể phát biểu lại bài toán như sau. Thứ nhất, nếu không tái sử dụng KV cache, round cuối đòi hỏi prefill lại tổng cộng gần 500 nghìn token ($20 \times \sim 24,9$ nghìn) dồn

trong nửa giây trên một lát MIG — TTFT đuôi chắc chắn vượt trần 1500 ms hàng loạt. Prefix caching biến chi phí mỗi round (từ round 2 trở đi) xuống chỉ còn khoảng 2,9 nghìn token mới cho mỗi request, tương đương ~58 nghìn token cho cả round. Nói cách khác, **prefix caching không phải một tối ưu tùy chọn mà là điều kiện tiên quyết để có điểm TTFT ở 100/120 request**. Thứ hai, round 1 là cold start không thể né: $20 \text{ request} \times \sim 12,9 \text{ nghìn token} = \sim 258 \text{ nghìn token}$ prefill dồn cục, không cache nào cứu được — đây chính là nơi các đội mạnh tách khỏi nhau, và là cuộc đua **thông lượng prefill thuần túy**. Thứ ba, cửa sổ TPOT hẹp (20–45 ms) với $w = 0,5$ nghĩa là một nửa số điểm của mọi request nằm ở việc giữ tốc độ decode ổn định $\geq 50 \text{ token/s/stream}$ ngay cả khi 20 stream decode đồng thời và có prefill chen ngang; đạt được Floor 20 ms thì không có điểm thưởng thêm khi nhanh hơn nữa, nên các kỹ thuật rủi ro như speculative decoding chỉ đáng dùng nếu đo được TPOT thực tế đang trên 20 ms. Thứ tư, vùng đệm accuracy rất rộng (được phép tụt 10 điểm phần trăm tuyệt đối, từ 0,40 xuống 0,30, mà không bị phạt) là lời mời gọi quantization mạnh tay có kiểm soát — FP8 weight gần như "miễn phí" về accuracy trong khi tăng tốc trực tiếp phần prefill vốn là nút thắt.

Ước lượng nhanh về ngân sách điểm để định vị mục tiêu (cần kiểm chứng bằng đo đạc thật, vì thông lượng prefill thực tế trên lát MIG là ẩn số quyết định): nếu prefix caching hoạt động tốt và thông lượng prefill đạt mức đưa TTFT trung bình các round 2–6 về vùng 300–700 ms, s_{ttft} trung bình của 100 request này rơi khoảng 0,4–0,7; giữ $s_{tpot} = 1$ cho toàn bộ; round 1 vớt được một phần TTFT của các request được xử lý sớm. Kịch bản đó cho Score vào khoảng 65–85 điểm với $f(\Delta) = 1$. Mỗi cải thiện thông lượng prefill dịch chuyển trực tiếp cả cụm điểm này lên, và với $\gamma = 2$, giá trị của từng ms cải thiện tăng dần khi tiến về gần Floor — nghĩa là chiến lược đúng là kéo **đa số** request về sát Floor, thay vì dồn sức cứu vài request tệ nhất đang nằm sát trần.

3.2. Các trục kỹ thuật theo thứ tự ưu tiên

Trục 1 — Xác minh và tối đa hóa prefix caching trên kiến trúc hybrid. Đây là việc số một tuần đầu. Baseline vLLM v0.22.1 đã bật enable-prefix-caching, nhưng Qwen3.5-2B là kiến trúc lai DeltaNet + attention, mà khả năng cache prefix cho các layer linear-attention (state hồi quy, không chia block như KV thường) trong từng engine là điểm cần kiểm chứng thực nghiệm chứ không được mặc định tin. Cách kiểm chứng: bắn lặp một hội thoại mở rộng dần (nội dung tự tạo, tuyệt đối không dùng nội dung trace) và so TTFT lượt sau với lượt đầu, đồng thời đọc

cache-hit metrics của engine. Nếu vLLM cache kém với kiến trúc này, SGLang với RadixAttention là phương án B phải thử sớm. Song song, tính ngân sách bộ nhớ để **bảo đảm không có eviction giữa các round**: cuối benchmark cần giữ đồng thời KV/state của 20 session $\times$ ~25 nghìn token; với kiến trúc lai, phần KV tăng theo chuỗi chỉ nằm ở số ít layer attention nên 18GB VRAM nhiều khả năng dư dả, nhưng phải tính chính xác bằng tokenizer và config thật của model. Hạ max-model-len từ 262144 xuống khoảng 32768 (đủ cho ~25 nghìn token prompt cộng 200 token output và phần đệm chat template) để engine phân bổ tài nguyên sát nhu cầu. Một điểm tinh tế cần lường trước: phần cache chắc chắn hit là prompt của lượt trước; message assistant trong trace do BTC sinh sẵn có thể không trùng token-by-token với những gì engine của mình sinh ra (nhất là sau quantization), nên không kỳ vọng cache hit vượt qua ranh giới đó — chi phí chuẩn mỗi round vẫn là ~2,9 nghìn token mới mỗi request.

Trục 2 — Tăng thông lượng prefill (quyết định round 1 và TTFT đầu các round sau).

FP8 weight quantization là bước có tỷ lệ lợi ích/rủi ro tốt nhất: tăng tốc GEMM phần prefill vốn compute-bound, trong khi mức tụt GPQA điển hình nằm sâu trong vùng đệm 10 điểm. Kết hợp chunked prefill và tinh chỉnh max-num-batched-tokens để cân bằng hai lực kéo ngược chiều: batched-tokens lớn giúp prefill tổng nhanh hơn (cứu TTFT đầu) nhưng làm decode-step phình ra (đe dọa TPOT của các stream đang decode). Đây là tham số cần sweep có hệ thống trên simulator nội bộ. INT4/AWQ chỉ cân nhắc nếu sau FP8 vẫn cần thêm tốc độ và GPQA nội bộ xác nhận còn margin ít nhất 5 điểm so với ngưỡng 0,30. KV cache FP8 để sau cùng và chỉ giữ nếu qua được cả hai cửa kiểm tra: không tụt GPQA và không phá cơ chế prefix cache trên kiến trúc lai.

Trục 3 — Bảo vệ TPOT dưới tải burst. Mục tiêu cứng: TPOT trung bình ≤ 20 ms cho toàn bộ 120 request. Với model 2B trên lát MIG, decode 20 stream đồng thời về lý thuyết nằm trong khả năng băng thông, nên rủi ro TPOT chủ yếu đến từ hai nguồn: prefill của round kế tiếp chen vào giữa lúc decode (kiểm soát bằng chunked prefill config ở Trục 2) và overhead phía CPU. Bật CUDA Graphs cho decode, kiểm tra TPOT ở đúng kịch bản xấu nhất (20 stream decode song song trong khi 20 prefill mới ập đến).

Trục 4 — Đừng xem nhẹ nút thắt CPU.

Instance chấm chỉ có 3 core CPU và 8GB RAM. Tokenize 20 prompt dài 13–25 nghìn token gần như đồng thời, cộng với overhead HTTP/streaming của 20 kết nối, hoàn toàn có thể cộng thêm hàng trăm ms vào TTFT một cách vô hình. Tắt log request, tối giản middleware, đảm bảo response streaming trả token thật sự tăng dần (nếu server buffer output, TTFT đo được sẽ tệ đi oan uổng).

Trục 5 — Đóng gói và vận hành

submission chuẩn chỉnh. Nướng sẵn trọng số vào image (luật cấm pull runtime), warmup lúc khởi động bằng request tự tạo (không lấy từ trace) để compile CUDA graphs và làm nóng allocator trước khi healthcheck pass, đo thời gian khởi động tổng để không rớt healthcheck trong cửa sổ chấm ~15 phút. Giữ nguyên tokenizer, không đung vào timestamp hay concurrency — mọi tối ưu phải nằm ở tầng engine.

3.3. Hạ tầng thực nghiệm và kỷ luật đo đạc

Tận dụng dàn GPU sẵn có theo phân vai đã xác định: H200 công ty (không MIG được) mô phỏng gần đúng môi trường chấm bằng cách cap bộ nhớ qua gpu-memory-utilization ≈ 0.128 và giới hạn compute qua CUDA MPS active thread percentage $\approx 14\%$ nếu daemon khả dụng — luôn nhớ rằng số latency tuyệt đối trên H200 đầy đủ sẽ lạc quan hơn thật nếu không giới hạn được SM; L40S cho thí nghiệm FP8; A30 cho quy trình MIG thật; RTX 3060 cho phát triển script. Hoàn thiện hai công cụ đã lên kế hoạch: simulator ERS nội bộ replay đúng trace với đúng timestamp và tính điểm theo đúng công thức (kể cả chi tiết TPOT_mean của vòng 1), và GPQA harness chạy trên toàn bộ GPQA Diamond làm proxy — vì 100 câu của BTC là cố định nhưng ẩn, mọi quyết định quantization phải giữ margin an toàn vài điểm so với ngưỡng 0,30 thay vì tối ưu sát mép. Kỷ luật thực nghiệm: mỗi lần chỉ thay một biến, ghi lại bảng config $\leftrightarrow$ điểm, và dùng leaderboard (phản hồi 15 phút) làm thước đo chân lý cuối cùng — nộp baseline sớm ngay tuần đầu để có mốc so sánh và xác nhận pipeline nộp bài trơn tru trước khi tối ưu sâu.

3.4. Lộ trình 25 ngày còn lại